

Document number: P1790R1
Date: 2020-01-13
Project: Programming Language C++
Audience: SG4 - Networking, LEWG
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to enable better DynamicBuffer composition

Introduction

P1100r0 *Efficient composition with DynamicBuffer* showed how the current specification of the DynamicBuffer type requirements inhibits layered composition of synchronous and asynchronous I/O operations. This new paper captures the LEWGI discussion of P1100r0 at the Kona 2019 meeting, wherein the root cause of the design issue was explored, and an alternative approach was discussed and accepted.

Background

The DynamicBuffer type requirements, and its supporting algorithms and implementations, have the following goals:

- To automatically grow to accommodate data
- To efficiently support delimited protocols
- To be adaptable to support circular buffers

A trivial use of a DynamicBuffer is illustrated in this example:

```
vector<unsigned char> data;  
// ...  
size_t n = net::read(my_socket,  
    net::dynamic_buffer(data, MY_MAX),  
    net::transfer_at_least(1));
```

The `net::dynamic_buffer` function creates a DynamicBuffer as a view on to the underlying memory associated with the vector `data`. The vector data is then automatically resized to accommodate newly received bytes, and following the `read` operation will contain these bytes.

n == 10	h	e	l	l	o	\n	a	b	c	d
---------	---	---	---	---	---	----	---	---	---	---

For delimited protocols, a typical use may look similar to the following example:

```
string data;  
// ...  
size_t n = net::read_until(my_socket  
    net::dynamic_buffer(data, MY_MAX),  
    '\n');
```

After `read_until` completes, the vector contains all newly received bytes, while `n` denotes the position of the first delimiter:

n == 6	h	e	l	l	o	\n	a	b	c	d
--------	---	---	---	---	---	----	---	---	---	---

Before issuing another `read_until` to obtain the next delimited record, the application should remove the first record from the buffer:

```
data.erase(0, n);
// issue next read_until ...
```

Similarly, use of a DynamicBuffer with a write operation will automatically shrink the underlying memory as the data from it is consumed:

```
size_t n = net::write(my_socket,
    net::dynamic_buffer(data));
// data is now empty
```

Both the statically sized buffer sequences, which are created by `net::buffer()`, and the dynamic buffers created by `net::dynamic_buffer`, should be considered as views on to some underlying memory. The difference between the two is that a dynamic buffer will resize the underlying memory as required.

To support this a DynamicBuffer is defined as follows:

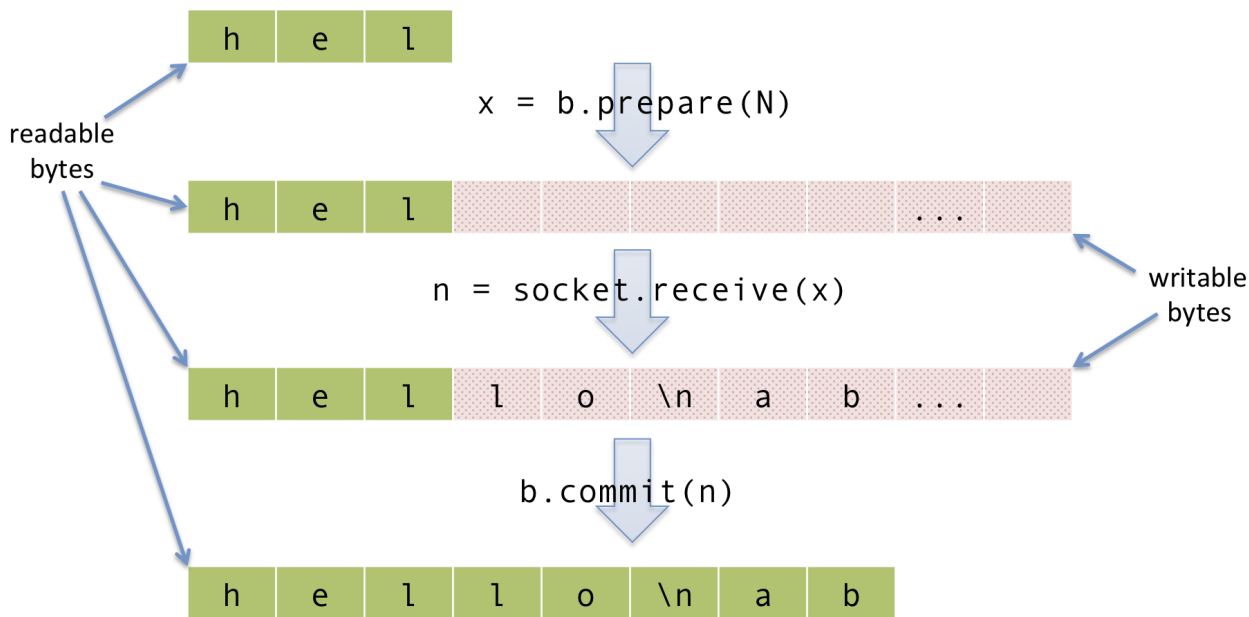
A dynamic buffer encapsulates memory storage that may be automatically resized as required, where the memory is divided into two regions: readable bytes followed by writable bytes. These memory regions are internal to the dynamic buffer, but direct access to the elements is provided to permit them to be efficiently used with I/O operations. [Note: Such as the send or receive operations of a socket. The readable bytes would be used as the constant buffer sequence for send, and the writable bytes used as the mutable buffer sequence for receive. —end note] Data written to the writable bytes of a dynamic buffer object is appended to the readable bytes of the same object.

A DynamicBuffer type is required to satisfy the requirements of Destructible and MoveConstructible, as well as the requirements shown in the table below.

expression	type	assertion/note pre/post-conditions
<code>X::const_buffers_type</code>	type meeting ConstBufferSequence requirements.	This type represents the memory associated with the readable bytes.
<code>X::mutable_buffers_type</code>	type meeting MutableBufferSequence requirements.	This type represents the memory associated with the writable bytes.
<code>x1.size()</code>	<code>size_t</code>	Returns the number of readable bytes.
<code>x1.max_size()</code>	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable, that can be held by <code>x1</code> .
<code>x1.capacity()</code>	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable, that can be held by <code>x1</code> without requiring reallocation.
<code>x1.data()</code>	<code>X::const_buffers_type</code>	Returns a constant buffer sequence <code>u</code> that represents the readable bytes, and where <code>buffer_size(u) == size()</code> .
<code>x.prepare(n)</code>	<code>X::mutable_buffers_type</code>	Returns a mutable buffer sequence <code>u</code> representing the writable bytes, and where <code>buffer_size(u) == n</code> . The dynamic buffer reallocates memory as required. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated. Throws: <code>length_error</code> if <code>size() + n</code> exceeds <code>max_size()</code> .

expression	type	assertion/note pre/post-conditions
<code>x.commit(n)</code>		Appends <i>n</i> bytes from the start of the writable bytes to the end of the readable bytes. The remainder of the writable bytes are discarded. If <i>n</i> is greater than the number of writable bytes, all writable bytes are appended to the readable bytes. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated.
<code>x.consume(n)</code>		Removes <i>n</i> bytes from beginning of the readable bytes. If <i>n</i> is greater than the number of readable bytes, all readable bytes are removed. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated.

This separation between the readable and writable bytes can be visualised in the following sequence of operations:



Historically, the separation of readable bytes and writable bytes originally followed the design model of `std::streambuf`, which divides its memory into input and output sequences. This design was preserved even though `DynamicBuffer` was ultimately decoupled from concrete `streambuf` classes.

Problem

Dynamic buffers are intended to be used in compositions, such as an algorithm that reads a sequence of delimited headers from a socket:

```
void read_headers(std::string& data)
{
    size_t n = net::read_until(my_socket, net::dynamic_buffer(data), '\n');
    // process 1st header and consume it
}
```

```

n = net::read_until(my_socket, net::dynamic_buffer(data), '\n');
// process 2nd header and consume it
}

```

However, a problem arises when we want to make our algorithm generic across all DynamicBuffer types:

```

template <typename DynamicBuffer>
void read_headers(DynamicBuffer buf)
{
    size_t n = net::read_until(my_socket, std::move(buf), '\n');
    // process 1st header and consume it
    n = net::read_until(my_socket, ???, '\n');
    // process 2nd header and consume it
}

```

As highlighted by P1100r0, we can see here that the current specification of the DynamicBuffer requirements inhibits layered composition of I/O operations. This is a consequence of the type requirements stipulating move-constructibility.

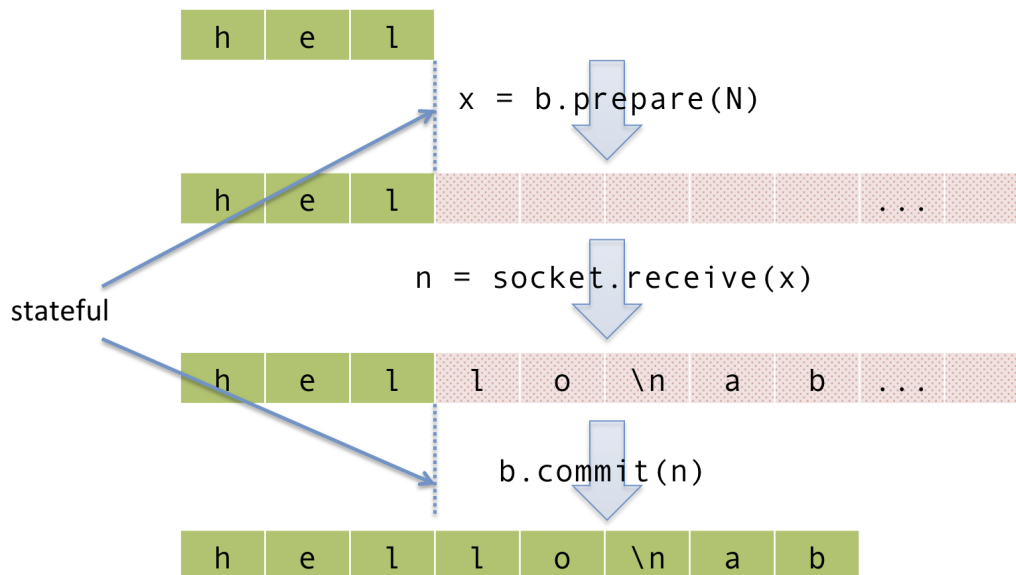
It is worth noting that this problem has no direct impact on the Networking TS itself, as DynamicBuffer is already sufficient for the needs of algorithms defined in the TS). However, we feel it is still worth addressing this problem to enable the development of higher layer algorithms that use dynamic buffers.

Analysis

The DynamicBuffer requirements embody two distinct responsibilities:

1. The ability to dynamically resize underlying memory regions.
2. To separate the buffer into two parts: readable bytes and writable bytes.

It is this second requirement that is the source of the problem, in that it requires a dynamic buffer class to be stateful. Specifically, the dynamic buffer has to maintain state, namely the boundary between the readable and writable regions of memory, across the operations that use it.



This statefulness can be observed in the implementation of concrete dynamic buffers, such as `dynamic_string_buffer`:

```

template <...>
class dynamic_string_buffer
{
    // ...
private:
    basic_string<...>& string_;
    size_t size_;
}

```

```
const size_t max_size_;
};
```

Solution proposed by P1100r0

P1100r0 proposed removing the `MoveConstructible` requirement from `DynamicBuffer`, and instead stipulating that `DynamicBuffer` types be used exclusively by reference.

This changes the typical use of a dynamic buffer as shown below:

```
string data;
// ...
size_t n = net::read_until(my_socket
    net::dynamic_buffer(data, MY_MAX),
    '\n');

net::dynamic_string_buffer data;
size_t n = net::read_until(my_socket, data, '\n');
std::string s = data.release();
```

but does enable the development of higher level abstractions:

```
template <typename DynamicBuffer>
void read_headers(DynamicBuffer& buf)
{
    size_t n = net::read_until(my_socket, std::move(buf)buf, '\n');
    // process 1st header and consume it
    n = net::read_until(my_socket, buf, '\n');
    // process 2nd header and consume it
}

// ...

net::dynamic_string_buffer data;
read_headers(data);
std::string s = data.release();
```

Alternate solution discussed and accepted by LEWGI

If we consider the way in which these two parts are used within networking TS I/O operations, we see that the distinction between readable and writable parts is actually only important for the duration of an operation.

For example, a `DynamicBuffer`-enabled read operation:

1. Prepares writable memory to be used as a target for the underlying read operation.
2. Performs the underlying operation.
3. Commits the number of bytes transferred by the operation.

Following the operation, the entire `DynamicBuffer` consists of readable bytes.

Thus, the alternative solution is to change the `DynamicBuffer` requirements to have one responsibility only:

- To dynamically resize the underlying memory.

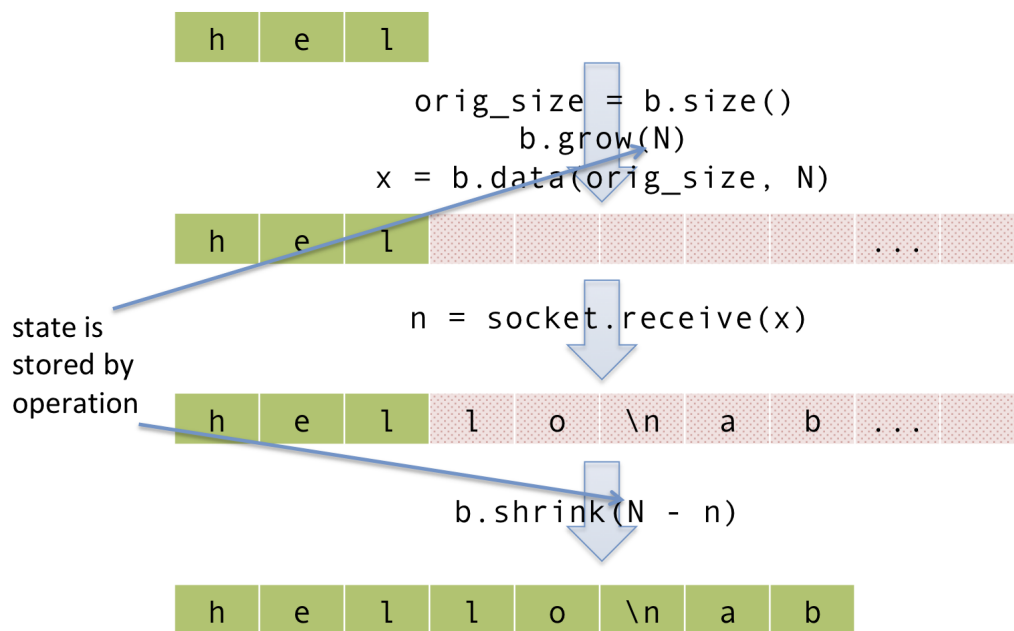
The responsibility for distinguishing between readable and writable bytes is moved to the operations and algorithms that work with `DynamicBuffer`. This removes the statefulness requirement from `DynamicBuffer`, and allows `DynamicBuffer` to be considered a lightweight, copy-constructible type (just as the statically sized `ConstBufferSequence` and `MutableBufferSequence` are).

Thus, the `DynamicBuffer` type requirements are changed from `MoveConstructible` to `CopyConstructible`, as well as the changes shown in the updated table below.

expression	type	assertion/note pre/post-conditions
<code>X::const_buffers_type</code>	type meeting ConstBufferSequence requirements.	This type represents the <u>underlying</u> memory associated with the readable bytes <u>as non-modifiable bytes</u> .
<code>X::mutable_buffers_type</code>	type meeting MutableBufferSequence requirements.	This type represents the <u>underlying</u> memory associated with the writable bytes <u>as modifiable bytes</u> .
<code>x1.size()</code>	<code>size_t</code>	Returns the number of readable bytes.
<code>x1.max_size()</code>	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable , that can be held by <code>x1</code> .
<code>x1.capacity()</code>	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable , that can be held by <code>x1</code> without requiring reallocation.
<code>x1.data()</code>	<code>X::const_buffers_type</code>	Returns a constant buffer sequence <code>u</code> that represents the readable bytes, and where <code>buffer_size(u) == size()</code>.
<code>x.prepare(n)</code>	<code>X::mutable_buffers_type</code>	Returns a mutable buffer sequence <code>u</code> representing the writable bytes, and where <code>buffer_size(u) == n</code>. The dynamic buffer reallocates memory as required. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated. Throws: <code>length_error</code> if <code>size() + n</code> exceeds <code>max_size()</code>.
<code>x.commit(n)</code>		Appends <code>n</code> bytes from the start of the writable bytes to the end of the readable bytes. The remainder of the writable bytes are discarded. If <code>n</code> is greater than the number of writable bytes, all writable bytes are appended to the readable bytes. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated.
<u><code>x1.data(pos, n)</code></u>	<u><code>X::const_buffers_type</code></u>	<u>Returns a constant buffer sequence <code>u</code> that represents the region of underlying memory at offset <code>pos</code> and length <code>n</code>.</u>
<u><code>x.data(pos, n)</code></u>	<u><code>X::mutable_buffers_type</code></u>	<u>Returns a mutable buffer sequence <code>u</code> that represents the region of underlying memory at offset <code>pos</code> and length <code>n</code>.</u>
<u><code>x.grow(n)</code></u>		<u>Adds <code>n</code> bytes of space at the end of the underlying memory. All constant or mutable buffer</u>

expression	type	assertion/note pre/post-conditions
		<u>sequences previously obtained using data(.) are invalidated.</u>
<u>x.shrink(n).</u>		<u>Removes n bytes of space from the end of the underlying memory. If n is greater than the number of bytes, all bytes are discarded. All constant or mutable buffer sequences previously obtained using data(.) are invalidated.</u>
x.consume(n)		Removes n bytes from beginning of the readable bytes. If n is greater than the number of readable bytes, all readable bytes are removed. All constant or mutable buffer sequences previously obtained using data() or prepare(.) are invalidated.

These new requirements are illustrated by the following sequence of operations:



Concrete dynamic buffer implementations such as `dynamic_string_buffer` now no longer need to maintain the state representing the marker between readable and writable bytes:

```
template <...>
class dynamic_string_buffer
{
    // ...
private:
    basic_string<...>& string_;
    size_t size_;
    const size_t max_size_;
};
```

Existing dynamic buffer uses are unchanged by this solution:

```
string data;
// ...
size_t n = net::read_until(my_socket,
    net::dynamic_buffer(data, MY_MAX),
    '\n');
```

and higher level abstractions are now possible:

```
template <typename DynamicBuffer>
void read_headers(DynamicBuffer buf)
{
    size_t n = net::read_until(my_socket, std::move(buf)buf, '\n');
    // process 1st header and consume it
    n = net::read_until(my_socket, buf, '\n');
    // process 2nd header and consume it
}

// ...

std::string data;
read_headers(net::dynamic_buffer(data));
```

A further advantage of this solution is that, by moving the distinction between the readable and writable bytes from the DynamicBuffer to the algorithms that use it, we now enable algorithms that need to update the notionally “committed” bytes. A simple example of this would be an algorithm that decodes base64-encoded data from a stream-based source: an additional byte of input may require an update to the final byte of output in the buffer. This capability has been requested by Asio users in the past.

Naming

We may wish to consider other options for the names grow, shrink, and consume. The author has no better suggestions to offer at this time.

Implementation experience

The accepted solution was implemented in Asio 1.14.0, which was delivered as part of the Boost 1.70 release.

Proposed changes to wording

In summary, the following changes would be made to the Networking TS wording:

- The DynamicBuffer requirements are changed as shown above.
- The dynamic_string_buffer and dynamic_vector_buffer classes are altered to satisfy these new requirements.
- The DynamicBuffer algorithms (read, async_read, read_until, async_read_until, write, and async_write) are changed to take DynamicBuffer objects by value, and to use the new grow and shrink member functions as required.

These changes are relative to N4771.

Update the <experimental/buffer> synopsis [buffer.synop] as follows:

```
// [buffer.read], synchronous read operations:
```

[...]

```
template<class SyncReadStream, class DynamicBuffer>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b);
template<class SyncReadStream, class DynamicBuffer>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncReadStream, class DynamicBuffer, class CompletionCondition>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b,
```



```

        CompletionCondition completion_condition);
template<class SyncReadStream, class DynamicBuffer, class CompletionCondition>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition, error_code& ec);

// [buffer.async.read], asynchronous read operations:

```

[...]

```

template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read(AsyncReadStream& stream,
        DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer,
    class CompletionCondition, class CompletionToken>
    DEDUCED async_read(AsyncReadStream& stream,
        DynamicBuffer&& b,
        CompletionCondition completion_condition,
        CompletionToken&& token);

```

```

// [buffer.write], synchronous write operations:

```

[...]

```

template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b);
template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition, error_code& ec);

```

```

// [buffer.async.write], asynchronous write operations:

```

[...]

```

template<class AsyncWriteStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
        DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncWriteStream, class DynamicBuffer,
    class CompletionCondition, class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
        DynamicBuffer&& b,
        CompletionCondition completion_condition,
        CompletionToken&& token);

```

```

// [buffer.read.until], synchronous delimited read operations:

```

```

template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, char delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
        char delim, error_code& ec);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, string_view delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
        string_view delim, error_code& ec);

```

```

// [buffer.async.read.until], asynchronous delimited read operations:

```

```
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, char delim,
                             CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, string_view delim,
                             CompletionToken&& token);
```

Update the DynamicBuffer requirements [buffer.reqmts.dynamicbuffer] as follows:

16.2.4 Dynamic buffer requirements

[buffer.reqmts.dynamicbuffer]

- 1 A dynamic buffer encapsulates memory storage that may be automatically resized as required, ~~where the memory is divided into two regions: readable bytes followed by writable bytes. These memory regions are internal to the dynamic buffer, but d~~Direct access to the elements is provided to permit them to be efficiently used with I/O operations. [*Note* : Such as the send or receive operations of a socket. ~~The readable bytes would be used as the constant buffer sequence for send, and the writable bytes used as the mutable buffer sequence for receive. — end note~~] Data written to the writable bytes of a dynamic buffer object is appended to the readable bytes of the same object.
- 2 A type X meets the DynamicBuffer requirements if it satisfies the requirements of Destructible (C++2014[destructible]) and ~~MoveConstructible (C++2014[moveconstructible])~~CopyConstructible (C++2014[copyconstructible]), as well as the additional requirements listed in Table 14.
- 3 In Table 14, x denotes a value of type X , $x1$ denotes a (possibly const) value of type X , pos denotes a (possibly const) value of type `size_t`, and n denotes a (possibly const) value of type `size_t`.

Table 14: DynamicBuffer requirements

expression	type	assertion/note pre/post-conditions
$X::$ <code>const_buffers_type</code>	type meeting ConstBufferSequence ([buffer.reqmts.constbuffersequence]) requirements.	This type represents the <u>underlying</u> memory associated with the readable bytes as <u>non-modifiable bytes</u> .
$X::$ <code>mutable_buffers_type</code>	type meeting MutableBufferSequence ([buffer.reqmts.constbuffersequence]) requirements.	This type represents the <u>underlying</u> memory associated with the writable bytes as <u>modifiable bytes</u> .
$x1.size()$	<code>size_t</code>	Returns the number of readable bytes.
$x1.max_size()$	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable , that can be held by $x1$.

<code>x1.capacity()</code>	<code>size_t</code>	Returns the maximum number of bytes, both readable and writable , that can be held by <code>x1</code> without requiring reallocation.
<code>x1.data()</code>	<code>X::const_buffers_type</code>	Returns a constant buffer sequence <code>u</code> that represents the readable bytes, and where <code>buffer_size(u) == size()</code>.
<code>x.prepare(n)</code>	<code>X::mutable_buffers_type</code>	Returns a mutable buffer sequence <code>u</code> representing the writable bytes, and where <code>buffer_size(u) == n</code>. The dynamic buffer reallocates memory as required. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated. <i>Throws:</i> <code>length_error</code> if <code>size() + n</code> exceeds <code>max_size()</code>.
<code>x.commit(n)</code>		Appends <code>n</code> bytes from the start of the writable bytes to the end of the readable bytes. The remainder of the writable bytes are discarded. If <code>n</code> is greater than the number of writable bytes, all writable bytes are appended to the readable bytes. All constant or mutable buffer sequences previously obtained using <code>data()</code> or <code>prepare()</code> are invalidated.

<code>x1.data(pos, n).</code>	<code>X::const_buffers_type</code>	Returns a constant buffer sequence <code>u</code> that represents the region of underlying memory at offset <code>pos</code> and length <code>n</code> .
<code>x.data(pos, n).</code>	<code>X::mutable_buffers_type</code>	Returns a mutable buffer sequence <code>u</code> that represents the region of underlying memory at offset <code>pos</code> and length <code>n</code> .
<code>x.grow(n).</code>		Adds <code>n</code> bytes of space at the end of the underlying memory. All constant or mutable buffer sequences previously obtained using <code>data(.)</code> are invalidated.
<code>x.shrink(n).</code>		Removes <code>n</code> bytes of space from the end of the underlying memory. If <code>n</code> is greater than the number of bytes, all bytes are discarded. All constant or mutable buffer sequences previously obtained using <code>data(.)</code> are invalidated.
<code>x.consume(n)</code>		Removes <code>n</code> bytes from beginning of the readable bytes. If <code>n</code> is greater than the number of readable bytes, all readable bytes are removed. All constant or mutable buffer sequences previously obtained using <code>data()</code> or prepare() are invalidated.

Modify the `dynamic_vector_buffer` class [`buffer.dynamic.vector`] as follows:

16.12 Class template `dynamic_vector_buffer`

- 1 Class template `dynamic_vector_buffer` is an adaptor [\[buffer.dynamic.vector\]](#) used to automatically grow or shrink a vector object, to reflect the data successfully transferred in an I/O operation.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class T, class Allocator>
class dynamic_vector_buffer
{
public:
    // types:
    using const_buffers_type = const_buffer;
    using mutable_buffers_type = mutable_buffer;

    // constructors:
    explicit dynamic_vector_buffer(vector<T, Allocator>& vec) noexcept;
    dynamic_vector_buffer(vector<T, Allocator>& vec,
                          size_t maximum_size) noexcept;
dynamic_vector_buffer(dynamic_vector_buffer&&) = default;

    // members:
    size_t size() const noexcept;
    size_t max_size() const noexcept;
    size_t capacity() const noexcept;
const_buffers_type data() const noexcept;
    mutable_buffers_type data(size_t pos, size_t n) noexcept;
    const_buffers_type data(size_t pos, size_t n) const noexcept;
mutable_buffers_type prepare(size_t n);
void commit(size_t n) noexcept;
    void grow(size_t n);
    void shrink(size_t n);
    void consume(size_t n);

private:
    vector<T, Allocator>& vec_; // exposition only
size_t size_; // exposition only
    const size_t max_size_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std
```

- 2 The `dynamic_vector_buffer` class template meets the requirements of `DynamicBuffer` ([\[buffer.reqmts.dynamicbuffer\]](#)).
- 3 The `dynamic_vector_buffer` class template requires that `T` is a trivially copyable or standard-layout type (C++2014[[basic.types](#)]) and that `sizeof(T) == 1`.

```
explicit dynamic_vector_buffer(vector<T, Allocator>& vec) noexcept;
```

- 4 *Effects:* Initializes `vec_` with `vec`, ~~`size_` with `vec.size()`~~; and `max_size_` with `vec.max_size()`.

```
dynamic_vector_buffer(vector<T, Allocator>& vec,
                      size_t maximum_size) noexcept;
```

```

5      Requires:vec.size() <= maximum_size.

6      Effects:Initializes vec_ with vec,size_ with vec.size(); and max_size_ with
      maximum_size.

      size_t size() const noexcept;

7      Returns:size_min(vec.size(), max_size_).

      size_t max_size() const noexcept;

8      Returns:max_size_.

      size_t capacity() const noexcept;

9      Returns:vec_.capacity()min(vec_.capacity(), max_size_).

const_buffers_type data() const noexcept;

10     Returns:buffer(vec_, size_);

mutable_buffers_type data(size_t pos, size_t n) noexcept;
const_buffers_type data(size_t pos, size_t n) const noexcept;

      Returns:buffer(buffer(vec_, max_size_) + pos, n).

mutable_buffers_type prepare(size_t n);

11     Effects:Performs vec_.resize(size_ + n);

12     Returns:buffer(buffer(vec_) + size_, n);

13     Remarks:length_error-if-size() + n-exceeds-max_size();

void commit(size_t n);

14     Effects:Performs:

        size_ += min(n, vec_.size() - size_);
        vec_.resize(size_);

void grow(size_t n);

      Effects:Performs vec_.resize(size() + n);.
      Throws:length_error if size() > max_size() || max_size() - size() < n.

void shrink(size_t n);

      Effects:Performs vec_.resize(n > size() ? 0 : size() - n);.

      void consume(size_t n);

15     Effects:Performs:

        size_t m = min(n, size_);
        vec_.erase(vec_.begin(), vec_.begin() + m);
        size_ -= m;
        vec_.erase(vec_.begin(), vec_.begin() + min(n, size()));.

```

Modify the `dynamic_string_buffer` class [`buffer.dynamic.string`] as follows:

16.13 Class template `dynamic_string_buffer`

- 1 Class template `dynamic_string_buffer` is an adaptor used [\[buffer.dynamic.string\]](#) to automatically grow or shrink a `basic_string` object, to reflect the data successfully transferred in an I/O operation.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class CharT, class Traits, class Allocator>
    class dynamic_string_buffer
    {
    public:
        // types:
        using const_buffers_type = const_buffer;
        using mutable_buffers_type = mutable_buffer;

        // constructors:
        explicit dynamic_string_buffer(basic_string<CharT, Traits, Allocator>& str) noexcept;
        dynamic_string_buffer(basic_string<CharT, Traits, Allocator>& str,
                               size_t maximum_size) noexcept;
dynamic_string_buffer(dynamic_string_buffer&&) = default;

        // members:
        size_t size() const noexcept;
        size_t max_size() const noexcept;
        size_t capacity() const noexcept;
const_buffers_type data() const noexcept;
        mutable_buffers_type data(size_t pos, size_t n) noexcept;
        const_buffers_type data(size_t pos, size_t n) const noexcept;
mutable_buffers_type prepare(size_t n);
void commit(size_t n) noexcept;
        void grow(size_t n);
        void shrink(size_t n);
        void consume(size_t n);

    private:
        basic_string<CharT, Traits, Allocator>& str_; // exposition only
size_t size_; // exposition only
        const size_t max_size_; // exposition only
    };

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std
```

- 2 The `dynamic_string_buffer` class template meets the requirements of `DynamicBuffer` ([\[buffer.reqmts.dynamicbuffer\]](#)).

- 3 The `dynamic_string_buffer` class template requires that `sizeof(CharT) == 1`.

```
explicit dynamic_string_buffer(basic_string<CharT, Traits, Allocator>& str) noexcept;
```

- 4 *Effects:* Initializes `str_` with `str`, ~~`size_` with `str.size()`~~, and `max_size_` with `str.max_size()`.

```
dynamic_string_buffer(basic_string<CharT, Traits, Allocator>& str,
                      size_t maximum_size) noexcept;
```

- 5 *Requires:* `str.size() <= maximum_size`.

```

6      Effects:Initializes str_ with str,size_ with str.size(); and max_size_ with
      maximum_size.

      size_t size() const noexcept;

7      Returns:size_min(str.size(), max_size_).

      size_t max_size() const noexcept;

8      Returns:max_size_.

      size_t capacity() const noexcept;

9      Returns:vec_.capacity();min(str.capacity(), max_size_).

const_buffers_type data() const noexcept;

10     Returns:buffer(str_, size_);

mutable_buffers_type data(size_t pos, size_t n) noexcept;
const_buffers_type data(size_t pos, size_t n) const noexcept;

      Returns:buffer(buffer(str_, max_size_) + pos, n).

mutable_buffers_type prepare(size_t n);

11     Effects:Performs str_.resize(size_ + n);

12     Returns:buffer(buffer(str_) + size_, n);

13     Remarks:length_error if size() + n exceeds max_size();

void commit(size_t n) noexcept;

14     Effects:Performs:

           size_ += min(n, str_.size() - size_);
           str_.resize(size_);

      void grow(size_t n);

           Effects:Performs str_.resize(size() + n);.
           Throws:length error if size() > max_size() || max_size() - size() < n.

      void shrink(size_t n);

           Effects:Performs str_.resize(n > size() ? 0 : size() - n);.

      void consume(size_t n);

15     Effects:Performs:

           size_t m = min(n, size_);
           str_.erase(0, m);
           size_ = m;
           str_.erase(0, min(n, size()));.

```

Modify the read function [buffer.read] as follows:

17.5 Synchronous read operations [buffer.read]

[...]


```

template<class SyncReadStream, class DynamicBuffer>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b);
template<class SyncReadStream, class DynamicBuffer>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncReadStream, class DynamicBuffer,
    class CompletionCondition>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition);
template<class SyncReadStream, class DynamicBuffer,
    class CompletionCondition>
    size_t read(SyncReadStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition,
        error_code& ec);

```

8 *Effects:* Clears `ec`, then reads data from the synchronous read stream ([`buffer.stream.reqmts.syncreadstream`]) object stream by performing zero or more calls to the stream's `read_some` member function.

9 Data is placed into the dynamic buffer ([`buffer.reqmts.dynamicbuffer`]) object `b`. A mutable buffer sequence ([`buffer.reqmts.mutablebuffersequence`]) `x` is obtained prior to each `read_some` call ~~using `b.prepare(N)`~~, by performing:

```

    auto orig_size = b.size();
    b.grow(N);
    auto x = b.data(orig_size, N);

```

where `N` is an unspecified value less than or equal to `b.max_size() - b.size()`. [*Note* : Implementations can use `b.capacity()` when determining `N`, to minimize the number of `read_some` calls performed on the stream. — *end note*] After each `read_some` call, the implementation performs ~~`b.commit(n)`~~`b.shrink(N - n)`, where `n` is the return value from `read_some`.

Modify the `async_read` function [`buffer.async.read`] as follows:

17.6 Asynchronous read operations [`buffer.async.read`]

[...]

```

template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read(AsyncReadStream& stream,
        DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer, class CompletionCondition,
    class CompletionToken>
    DEDUCED async_read(AsyncReadStream& stream,
        DynamicBuffer&& b,
        CompletionCondition completion_condition,
        CompletionToken&& token);

```

9 *Completion signature:* `void(error_code ec, size_t n)`.

10 *Effects:* Initiates an asynchronous operation to read data from the buffer-oriented asynchronous read stream ([`buffer.stream.reqmts.asyncreadstream`]) object stream by performing one or more asynchronous `read_some` operations on the stream.

11 Data is placed into the dynamic buffer ([`buffer.reqmts.dynamicbuffer`]) object `b`. A mutable buffer sequence ([`buffer.reqmts.mutablebuffersequence`]) `x` is obtained prior to each `async_read_some` call ~~using `b.prepare(N)`~~, by performing:

```

auto orig_size = b.size();
b.grow(N);
auto x = b.data(orig_size, N);

```

where `N` is an unspecified value such that `N` is less than or equal to `b.max_size() - b.size()`. [*Note* : Implementations can use `b.capacity()` when determining `N`, to minimize the number of asynchronous `read_some` operations performed on the stream. — *end note*] After the completion of each asynchronous `read_some` operation, the implementation performs `b.commit(n)b.shrink(N - n)`, where `n` is the value passed to the asynchronous `read_some` operation's completion handler.

Modify the `write` function [`buffer.write`] as follows:

17.7 Synchronous write operations [`buffer.write`]

[...]

```

template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b);
template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
                 CompletionCondition completion_condition);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
                 CompletionCondition completion_condition,
                 error_code& ec);

```

Modify the `async_write` function [`buffer.async.write`] as follows:

17.8 Asynchronous write operations [`buffer.async.write`]

[...]

```

template<class AsyncWriteStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
                        DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncWriteStream, class DynamicBuffer, class CompletionCondition,
        class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
                        DynamicBuffer&& b,
                        CompletionCondition completion_condition,
                        CompletionToken&& token);

```

Modify the `read_until` function [`buffer.read.until`] as follows:

17.9 Synchronous delimited read operations [`buffer.read.until`]

```

template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, char delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                     char delim, error_code& ec);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, string_view delim);
template<class SyncReadStream, class DynamicBuffer>

```

```
size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                 string_view delim, error_code& ec);
```

1 *Effects:* Reads data from the buffer-oriented synchronous read stream ([[buffer.stream.reqmts.syncreadstream](#)]) object stream by performing zero or more calls to the stream's `read_some` member function, until ~~the readable bytes of~~ the dynamic buffer ([[buffer.reqmts.dynamicbuffer](#)]) object `b` contains the specified delimiter `delim`.

2 Data is placed into the dynamic buffer object `b`. A mutable buffer sequence ([[buffer.reqmts.mutablebuffersequence](#)]) `x` is obtained prior to each `read_some` call ~~using `b.prepare(N)`~~; by performing:

```
auto orig_size = b.size();
b.grow(N);
auto x = b.data(orig_size, N);
```

where `N` is an unspecified value such that `N <= max_size() - size()`. [*Note* : Implementations can use `b.capacity()` when determining `N`, to minimize the number of `read_some` calls performed on the stream. — *end note*] After each `read_some` call, the implementation performs ~~`b.commit(n)`~~ `b.shrink(N - n)`, where `n` is the return value from `read_some`.

3 The synchronous `read_until` operation continues until:

(3.1) — ~~the readable bytes of~~ `b` contains the delimiter `delim`; or

(3.2) — `b.size() == b.max_size()`; or

(3.3) — an asynchronous `read_some` operation fails.

4 On exit, if ~~the readable bytes of~~ `tcodes` contains the delimiter, `ec` is set such that `!ec` is `true`. Otherwise, if `b.size() == b.max_size()`, `ec` is set such that `ec == stream_errc::not_found`. If `b.size() < b.max_size()`, `ec` contains the `error_code` from the most recent `read_some` call.

5 *Returns:* The number of bytes in ~~the readable bytes of~~ `b` up to and including the delimiter, if present. [*Note* : On completion, the buffer can contain additional bytes following the delimiter. — *end note*] Otherwise returns `0`.

Modify the `async_read_until` function [[buffer.async.read.until](#)] as follows:

17.10 Asynchronous delimited read operations

[[buffer.async.read.until](#)]

```
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, char delim,
                             CompletionToken&& token);

template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, string_view delim,
                             CompletionToken&& token);
```

1 A composed asynchronous operation ([[async.reqmts.async.composed](#)]).

2 *Completion signature:* `void(error_code ec, size_t n)`.

3 *Effects:* Initiates an asynchronous operation to read data from the buffer-oriented asynchronous read stream ([[buffer.stream.reqmts.asyncreadstream](#)]) object stream by performing zero or more asynchronous `read_some` operations on the stream, until ~~the~~

~~readable bytes of~~ the dynamic buffer ([`buffer.reqmts.dynamicbuffer`]) object `b` contains the specified delimiter `delim`.

- 4 Data is placed into the dynamic buffer object `b`. A mutable buffer sequence ([`buffer.reqmts.mutablebuffersequence`]) `x` is obtained prior to each `async_read_some` call ~~using `b.prepare(N)`~~, by performing:

```
auto orig_size = b.size();  
b.grow(N);  
auto x = b.data(orig_size, N);
```

where `N` is an unspecified value such that `N <= max_size() - size()`. [*Note* : Implementations can use `b.capacity()` when determining `N`, to minimize the number of asynchronous `read_some` operations performed on the stream. — *end note*] After the completion of each asynchronous `read_some` operation, the implementation performs ~~`b.commit(n)`~~`b.shrink(N - n)`, where `n` is the value passed to the asynchronous `read_some` operation's completion handler.

- 5 The asynchronous `read_until` operation continues until:

- (5.1) — ~~the readable bytes of~~ `b` contains the delimiter `delim`; or
- (5.2) — `b.size() == b.max_size()`; or
- (5.3) — an asynchronous `read_some` operation fails.

- 6 The program shall ensure the `AsyncReadStream` object `stream` is valid until the completion handler for the asynchronous operation is invoked.

- 7 If `delim` is of type `string_view`, the implementation copies the underlying sequence of characters prior to initiating an asynchronous `read_some` operation on the stream. [*Note* : This means that the caller is not required to guarantee the validity of the delimiter string after the call to `async_read_until` returns. — *end note*]

- 8 On completion of the asynchronous operation, if ~~the readable bytes of~~ `b` contains the delimiter, `ec` is set such that `!ec` is `true`. Otherwise, if `b.size() == b.max_size()`, `ec` is set such that `ec == stream_errc::not_found`. If `b.size() < b.max_size()`, `ec` is the `error_code` from the most recent asynchronous `read_some` operation. `n` is the number of readable bytes in `b` up to and including the delimiter, if present, otherwise `0`.

History

- 2020-01-13: R1: Added detailed wording.
- Belfast 2019-11, SG4: Consensus to forward a revision with wording to LEWG.
- Cologne 2019-07, LEWGI: Consensus to forward to LEWG.
- 2019-09-16, R0: Initial revision.